

CoEvolution of Coding Agents, Code and Human by Evaluation

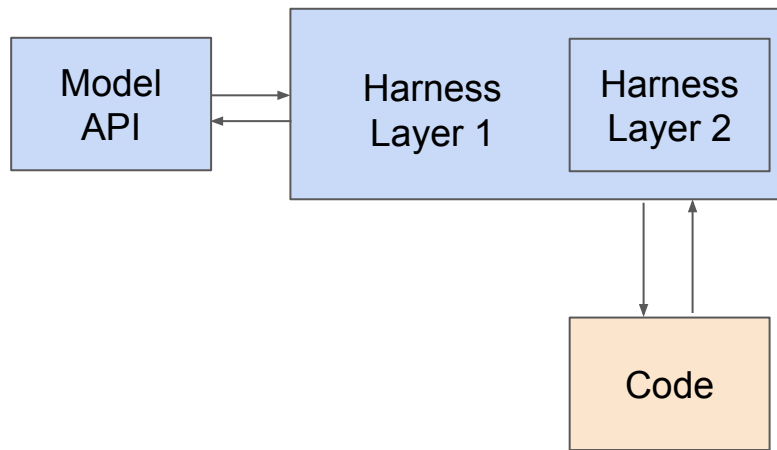
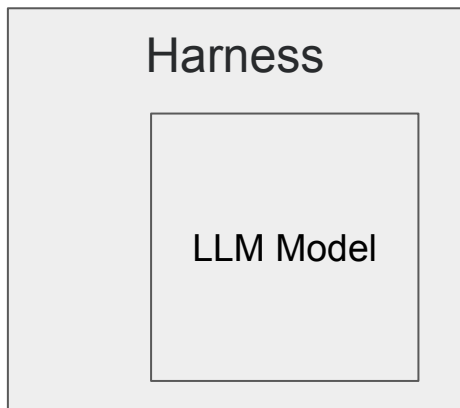
김기범, 2026.02.21



Coding Agent의 구조

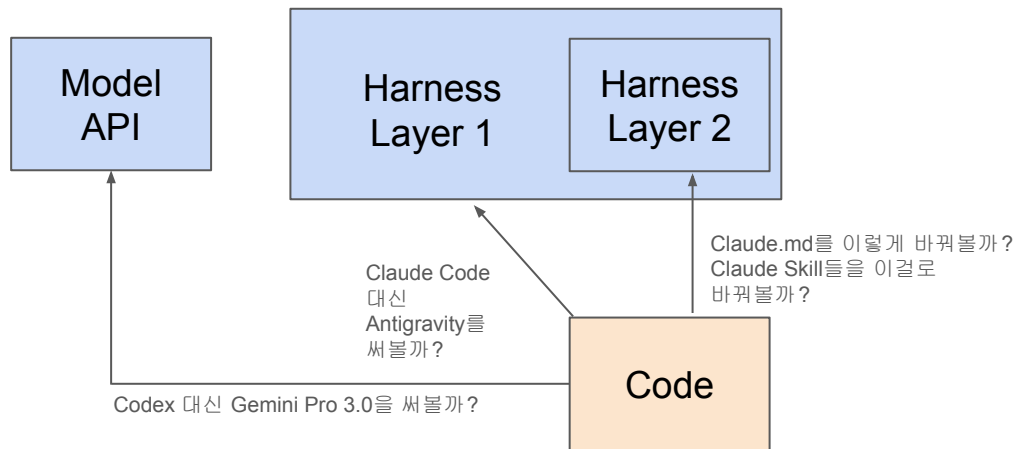
Harness: Model을 감싸는 도구의 총체

1. agent tools: claude code, antigraivity, codex 등
2. claude.md, Skills, system prompt 등



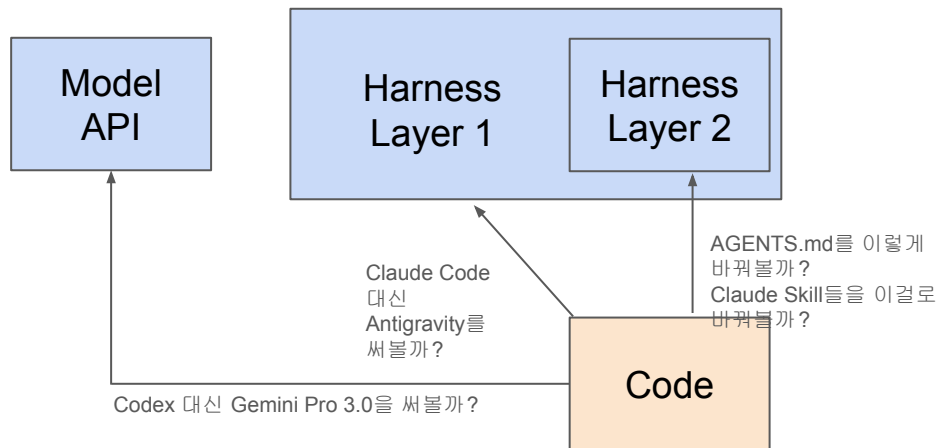
Agent Tuning

왠지 잘 안 돌아가는 것
같은데?



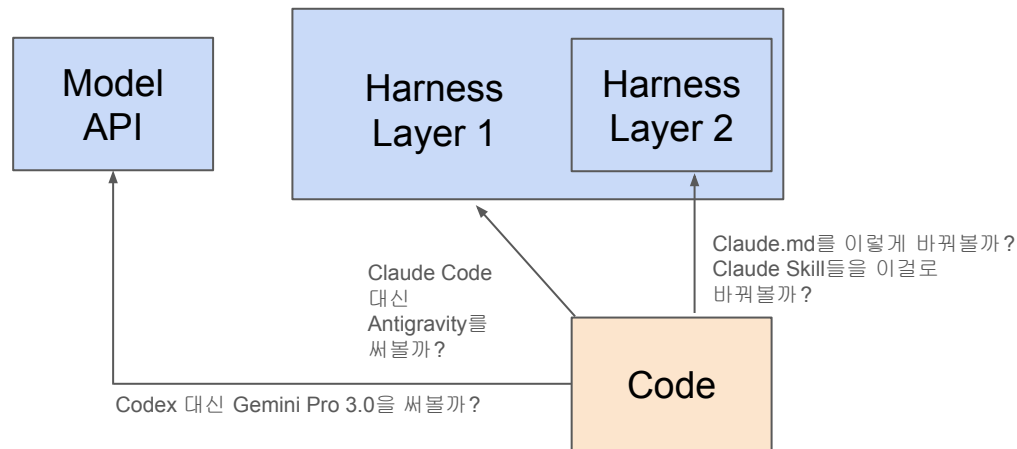
Agent Tuning시 고려해야 할 점들

- **Quality:** 더 성능이 좋은가?
- **Speed:** 더 빠르게 개발이 가능한가?
- **Cost:** 더 저렴하게 개발이 가능한가?
- **Tuning Cost:** 지금 내가 Tuning에 시간을 쓰는 것이 비용 대비 가치가 높은가?



Agent Tuning시 고려하는 것들

FEELING



“어느 **Guru**가 이야기하길, 이렇게 쓰면 좋대”

“며칠 사용해보니 이런 면에서 **Agent**가 미진하다는 인상을 받았어.”

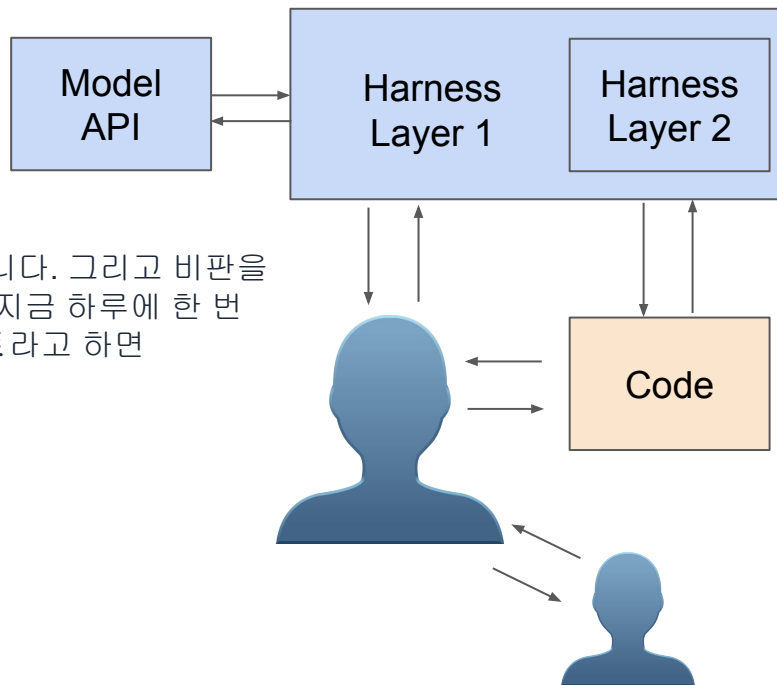
Agent Tuning 예시

“최근 연구 결과들에 의하면 애가 자기가 테스트되고 있는 환경이라는 걸 컨텍스트를 파악하는 능력 때문에 여러 종류의 테스트들이 실패한다는 연구 결과들이 있거든요. 그래서 방어적이 되지 않도록, 지금 시키는 일들이 야 너를 고치려는 게 아니라 너랑 같이 일할 다른 애들에게 줄 데이터다 라는 거를 은연중에 암시를 하는 식으로 글을 적고 있어요.”

“서브 에이전트는 서브 에이전트를 call할 수가 없거든요. 작년 초 중순까지 됐다가 그게 무한 루프 만드는 경우가 많아가지고 없어졌어요.”

“초창기부터 저는 존댓말을 썼는데요. AI한테 반말을 하기 시작하면 제가 사람에게 반말을 하게 될 수도 있죠.”

“제 핵심은 여러 각도에서의 비판을 스스로 시키는 겁니다. 그리고 비판을 스스로 시켜서 그 결과를 바꾸는 게 아니에요. 결국엔 지금 하루에 한 번 또는 두 번씩 돌아가고 있는 harness의 자기 업데이트 라고 하면 되겠네요.”



Agent Tuning 예시

“최근 연구 결과들에 의하면 애가 자기가 테스트되고 있는 환경이라는 걸 컨텍스트를 파악하는 능력 때문에 여러 종류의 테스트들이 실패한다는 연구 결과들이 있거든요. 그래서 방어적이 되지 않도록, 지금 시키는 일들이 야 너를 고치려는 게 아니라 너랑 같이 일할 다른 애들에게 줄 데이터다 라는 거를 은연중에 암시를 하는 식으로 글을 적고 있어요.”

“서브 에이전트는 서브 에이전트를 call할 수가 없거든요. 작년 초 중순까지 됐다가 그게 무한 루프 만드는 경우가 많아가지고 없어졌어요.”

“초창기부터 저는 존댓말을 썼는데요. AI한테 반말을 하기 시작하면 제가 사람에게 반말을 하게 될 수도 있죠.”

“제 핵심은 여러 각도에서의 비판을 스스로 시키는 겁니다. 그리고 비판을 스스로 시켜서 그 결과를 바꾸는 게 아니에요. 결국엔 지금 하루에 한 번 또는 두 번씩 돌아가고 있는 **harness**의 자기 업데이트라고 하면 되겠네요.”

1. Coding Agent를 사람 다루듯 대한다.

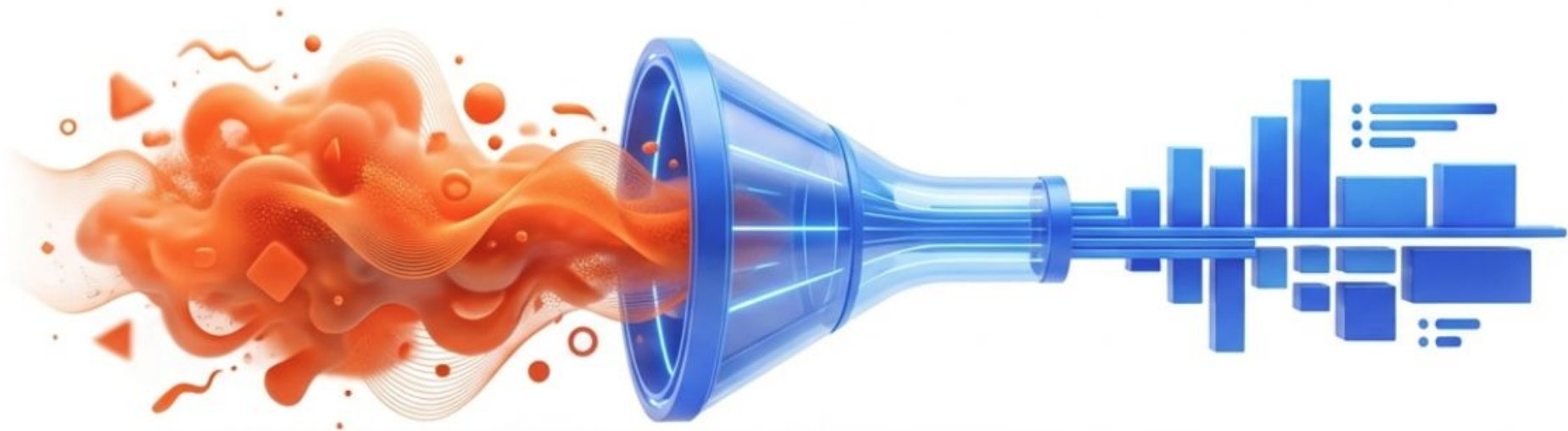
2. Agent Tuning에도 암묵지가 기능한다.

But, 정량적 측정이 없으면 발전 속도가 더디다.

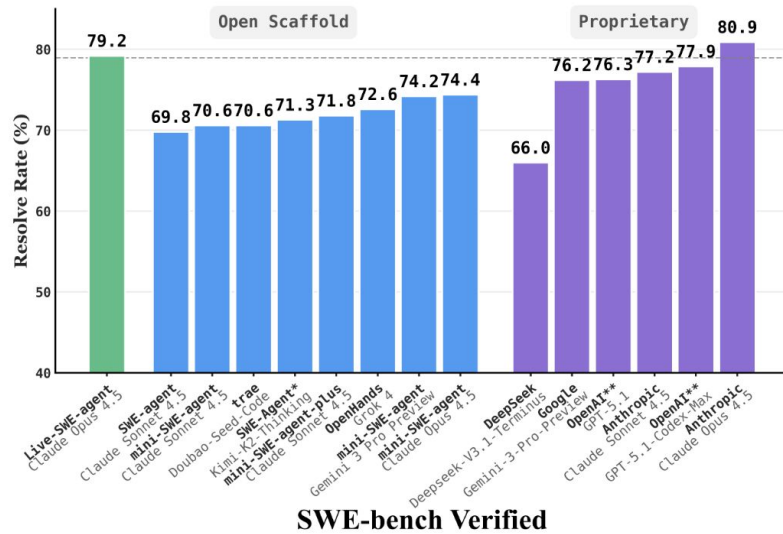
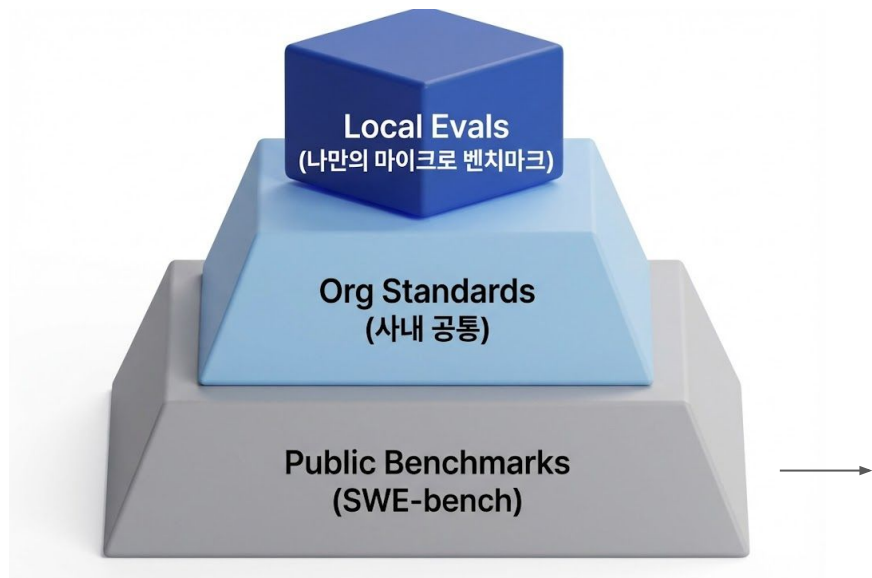
Vibe (느낌)

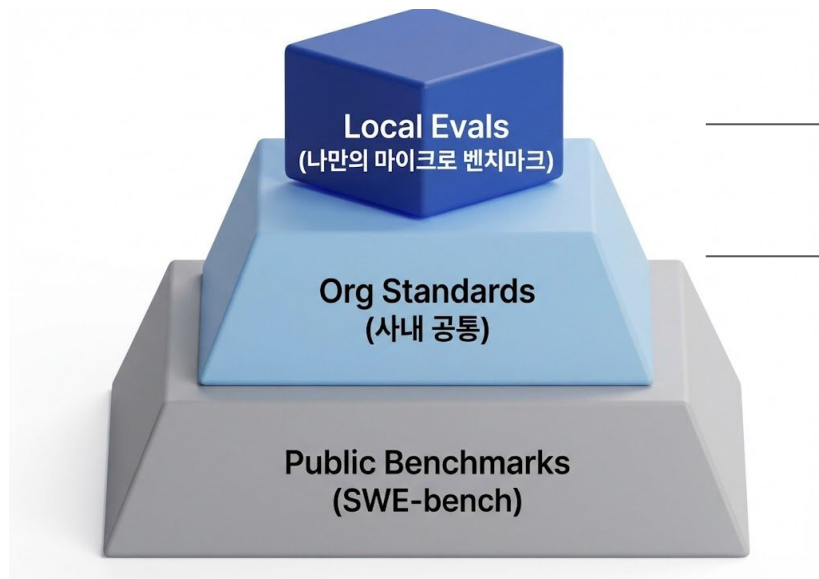
Evaluation (평가)

Data (데이터)



Agent 평가의 3단계





Intra-Project: 개인, 직무에 따른 Coding Agent 개선
Inter-Project: Single Trial마다 output을 테스트 구조

Organization-Aligned

Improving Deep Agents with harness engineering

- Build & Self-Verify 루프 강제

가장 흔한 실패 패턴: '코드 작성 → 자기 코드 다시 읽기 → 그럴듯하네 → 종료', 즉 테스트/검증 없이 멈추는 것
시스템 프롬프트에 문제 해결 절차를 명시적으로 넣기.

Planning & Discovery → Build → **Verify** → Fix

프롬프트만으로 '종료 직전 검증' 작동 한계. Middleware를 두어 에이전트가 종료하려는 순간 '검증 실행'을 리마인드.

- 불필요하게 긴 루프 제거

에이전트가 한 번 계획을 세우면 근시안적으로 고집, 같은 파일을 10번 이상 조금씩만 바꾸는 'doom loop'가 발생.

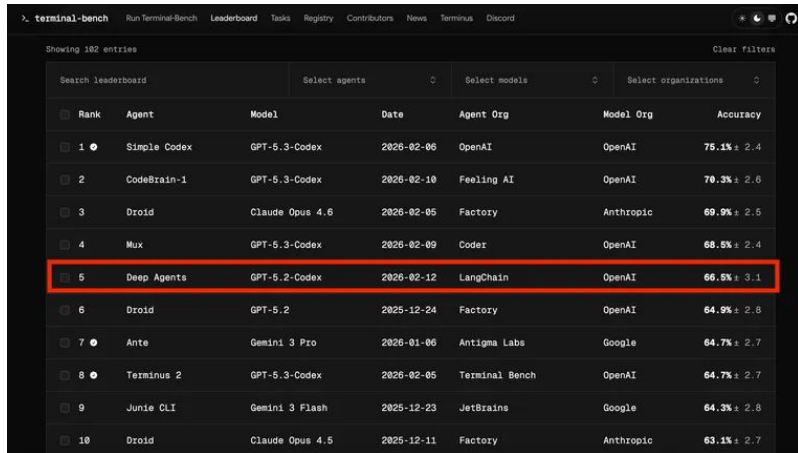
Middleware가 파일별 edit 횟수를 추적하다가 임계치(N) 이후 "접근을 재고하라"는 컨텍스트를 주입.

Terminal bench 30th→5th

- 환경 컨텍스트를 Rule based로 주입(context 절약)

- Reasoning sandwich: 태스크 단계별로 다른 reasoning 모드

* Middleware: rule based



Rank	Agent	Model	Date	Agent Org	Model Org	Accuracy
1	Simple Codex	GPT-5.3-Codex	2026-02-06	OpenAI	OpenAI	75.1% ± 2.4
2	CodeBrain-1	GPT-5.3-Codex	2026-02-10	Feeling AI	OpenAI	70.3% ± 2.6
3	Droid	Claude Opus 4.6	2026-02-05	Factory	Anthropic	69.9% ± 2.5
4	Mux	GPT-5.3-Codex	2026-02-09	Coder	OpenAI	68.6% ± 2.4
5	Deep Agents	GPT-5.2-Codex	2026-02-12	LangChain	OpenAI	66.6% ± 3.1
6	Droid	GPT-5.2	2025-12-24	Factory	OpenAI	64.9% ± 2.8
7	Ante	Gemini 3 Pro	2026-01-06	Antigma Labs	Google	64.7% ± 2.7
8	Terminus 2	GPT-5.3-Codex	2026-02-05	Terminal Bench	OpenAI	64.7% ± 2.7
9	Junie CLI	Gemini 3 Flash	2025-12-23	JetBrains	Google	64.3% ± 2.8
10	Droid	Claude Opus 4.5	2025-12-11	Factory	Anthropic	63.1% ± 2.7

Improving Deep Agents with harness engineering

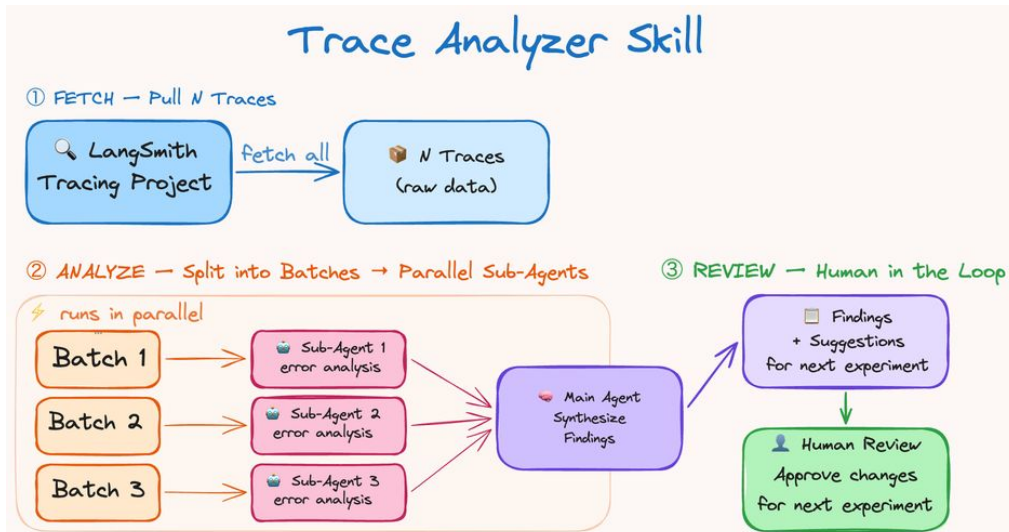
Trace Analyzer Skill

- 실패 모드 분석기.
- LangSmith에서 실험 트레이스 수집
- 병렬 에러 분석 에이전트들을 띄워 패턴을 찾고, 메인 에이전트가 종합/제안

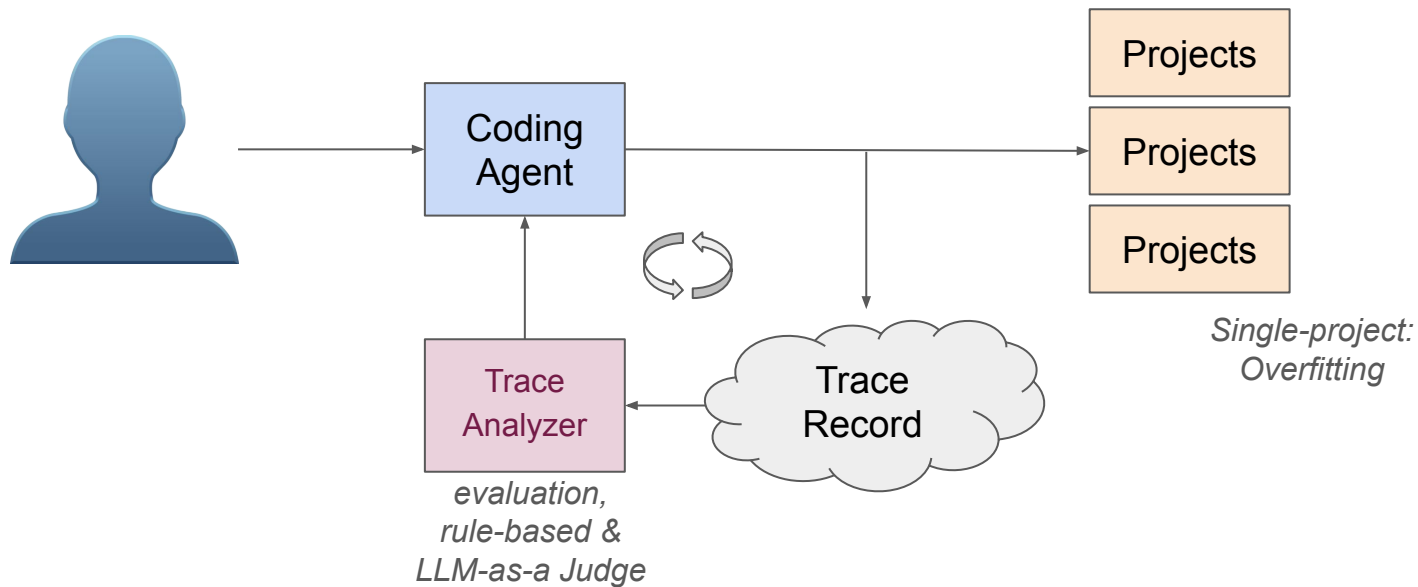
한계:

Terminal Bench Overfitting이다.

harness engineering 전/후 다른 문제들로 확인했어야 함.



Local Eval에 Trace Analyzer Skill를 적용한다면?



Open Questions

- 개발자 입장에서 (coding)Agent/Harness Tuning에 얼마나 많은 시간을 들여야 할까?
- Human Resource(HR) Manager의 일이 줄고, Agent Resource Engineer이라는 역할이 등장할까?

참고할만한 문서들

Harness Engineering, OpenAI, 2026.02

<https://openai.com/index/harness-engineering/>

- Evaluation harnesses → 메트릭, 품질등급을 지속적으로 측정

Effective harnesses for long-running agents, Anthropic, 2025.11

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

- 테스트 없이는 완료 처리 금지

Closing the Loop: Coding Agents, Telemetry, and the Path to Self-Improving Software

<https://arize.com/blog/closing-the-loop-coding-agents-telemetry-and-the-path-to-self-improving-software/>

- 코드가 아니라 트레이스가 문서다: 트레이스는 에이전트가 몇 번 루프를 돌았는지, 어떤 툴을 호출했는지, 어디서 실패가 났는지를 기록